

Лабораторна робота №3

Діаграми UML для системи
«Ізолятор тимчасового утримання
громадян-порушників»

Варіант №30

Лабораторна робота складається з чотирьох частин:

- Ч.1 — Діаграма класів (Class Diagram)
- Ч.2 — Діаграми станів та переходів (Statechart Diagrams)
- Ч.3 — Діаграми діяльності (Activity Diagrams)
- Ч.4 — Діаграма компонентів (Component Diagram)

Частина 1. Діаграма класів (Class Diagram)

Система включає 15 класів. Нижче наведено кожен клас із переліком атрибутів та операцій у UML-нотації, а також перелік відношень між класами.

«abstract»
Person
id : UUID # lastName : String # firstName : String # middleName : String # birthDate : Date # gender : String # documentNumber : String # address : String
+ getFullName() : String + getAge() : Integer + validateDocument() : Boolean + getDocumentInfo() : String + updateAddress(address: String) : void
Detainee
- caseNumber : String - photoPath : String - healthNotes : String - specialMarks : String - currentStatus : String
+ register(record: DetentionRecord) : void + getCurrentDetention() : DetentionRecord + getDetentionHistory() : List + generateBarcode() : String + updateHealthNotes(notes: String) : void

«abstract»
Staff
employeeld : String # username : String # passwordHash : String # role : String # isActive : Boolean
+ login(username: String, password: String) : Boolean + logout() : void + changePassword(old: String, new: String) : Boolean + getRole() : String + checkPermission(action: String) : Boolean

DutyOfficer
- shiftStart : DateTime - shiftEnd : DateTime - isOnShift : Boolean - badgeNumber : String - supervisorId : UUID
+ startShift() : void + endShift() : void + registerDetainee(d: Detainee) : DetentionRecord + processRelease(r: DetentionRecord) : void + addEvent(e: Event) : void

MedicalStaff
- licenseNumber : String - specialization : String - availableFrom : Time - availableTo : Time - qualification : String
+ conductExamination(d: Detainee) : Event + updateHealthRecord(d: Detainee, notes: String) : void + prescribeTreatment(d: Detainee, t: String) : void + checkMedicalHistory(d: Detainee) : String + issueHealthCertificate(d: Detainee) : Document

DetentionRecord
- id : UUID - protocolNumber : String - article : String - arrivalTime : DateTime - legalDeadline : DateTime - releaseTime : DateTime - status : String
+ calculateDeadline(hours: Integer) : DateTime + getRemainingTime() : Duration + extendDetention(reason: String, newDeadline: DateTime) : void + release(reason: String) : void + transfer(destination: String) : void

Cell
- id : Integer - cellNumber : String - capacity : Integer - genderType : String - isMedical : Boolean - currentOccupancy : Integer
+ getCurrentOccupants() : List + isAvailable() : Boolean + getOccupancyRate() : Double + addDetainee(a: CellAssignment) : void + removeDetainee(id: UUID) : void

CellAssignment
- id : UUID - detentionId : UUID - cellId : Integer - assignedAt : DateTime - removedAt : DateTime - transferReason : String
+ assign(cell: Cell, dr: DetentionRecord) : void + remove(reason: String) : void + getDuration() : Duration + isActive() : Boolean + getHistory() : List

Event
- id : UUID - detentionId : UUID - eventType : String - eventTime : DateTime - description : String - participantName : String
+ register(dr: DetentionRecord) : void + getDetails() : String + validate() : Boolean + generateReport() : Document + getParticipantInfo() : String

Document
- id : UUID - documentType : String - createdAt : DateTime - createdBy : UUID - filePath : String - isSigned : Boolean
+ generate(dr: DetentionRecord) : void + print() : void + saveToPdf() : void + sign(officer: Staff) : void + archive() : void

Property
- id : UUID - detentionId : UUID - itemName : String - description : String - storedAt : DateTime - isReturned : Boolean
+ confiscate(dr: DetentionRecord) : void + returnToOwner(officer: DutyOfficer) : void + generateReceipt() : Document + getStorageInfo() : String + updateDescription(desc: String) : void

AuditLog
- id : Long - userId : UUID - action : String - tableName : String - oldValues : String - performedAt : DateTime - ipAddress : String
+ logAction(user: Staff, action: String) : void + getLogsByUser(userId: UUID) : List + getLogsByDate(from: Date, to: Date) : List + exportToExcel() : void + searchLogs(query: String) : List
«interface»
INotificationService
+ alertThresholdHours : Integer + soundEnabled : Boolean + repeatInterval : Integer + maxAlertsPerHour : Integer + notificationLog : List
+ sendAlert(dr: DetentionRecord) : void + checkDeadlines() : void + scheduleReminder(dr: DetentionRecord, delay: Integer) : void + cancelReminder(id: UUID) : void + getActiveAlerts() : List
ReportGenerator
- reportType : String - dateFrom : Date - dateTo : Date - outputFormat : String - templatePath : String
+ generateStatistics(from: Date, to: Date) : Document + generateDetaineeCard(d: Detainee) : Document + generateReleaseAct(dr: DetentionRecord) : Document + exportToExcel(data: List) : void + printDocument(doc: Document) : void

«parameterized»
Repository<T>
- connectionString : String - tableName : String - timeout : Integer - pageSize : Integer - isConnected : Boolean
+ save(entity: T) : Boolean + findById(id: UUID) : T + findAll(filter: String) : List<T> + update(entity: T) : Boolean + delete(id: UUID) : Boolean

Відношення між класами

Тип відношення	Клас А	Зв'язок	Клас В	Пояснення
Наслідування (Generalization)	Person	◁---	Detainee	Затриманий є підтипом Person
Наслідування (Generalization)	Person	◁---	Staff	Персонал є підтипом Person
Наслідування (Generalization)	Staff	◁---	DutyOfficer	Черговий є підтипом Staff
Наслідування (Generalization)	Staff	◁---	MedicalStaff	Медпрацівник є підтипом Staff
Асоціація (Association)	Detainee	1 ---- 0..*	DetentionRecord	Одна особа — кілька фактів утримання
Асоціація (Association)	DutyOfficer	1 ---- 0..*	DetentionRecord	Черговий реєструє утримання
Композиція (Composition)	DetentionRecord	1 ◆---- 1..*	Event	Події існують тільки з утриманням
Композиція (Composition)	DetentionRecord	1 ◆---- 0..*	Property	Майно прив'язане до справи
Агрегація (Aggregation)	DetentionRecord	0..* ◇---- 1..*	Document	Документи можуть існувати окремо
Агрегація (Aggregation)	Cell	1 ◇---- 0..*	CellAssignment	Розміщення залежить від камери
Асоціація (Association)	CellAssignment	----	DetentionRecord	Прив'язка до конкретного утримання
Асоціація (Association)	AuditLog	----	Staff	Журнал фіксує дії персоналу
Реалізація (Realization)	ReportGenerator	--▷	INotificationService	Реалізує інтерфейс сповіщень
Інстанціювання (Instantiation)	Repository<T>	--→	Repository<DetentionRecord>	Шаблон з конкретним типом
Асоціація (Association)	ReportGenerator	---- 1..*	Document	Генератор створює документи

Частина 2. Діаграми станів та переходів (Statechart Diagrams)

Діаграма 2.1 — Функціонування системи ІТУ в цілому

Діаграма описує повний життєвий цикл роботи програмного комплексу від запуску до вимкнення.

Стан	Внутрішні дії	Тип
Вимкнена	—	Початковий
Ініціалізація (Initializing)	entry/ Завантаження конфігурації do/ Підключення до БД	Звичайний
Очікування (Idle)	entry/ Вибір функції меню do/ Вибір функції меню do/ Моніторинг строків (таймер) do/ Очікування дій користувача	Звичайний
Обробка запиту (Processing)	entry/ Валідація вхідних даних do/ Виконання операції do/ Запис у журнал аудиту	Звичайний
Формування звіту (Reporting)	entry/ Обробка даних з сервера do/ Генерація документа exit/ Відправка на принтер / збереження PDF	Звичайний
Резервне копіювання (Backup)	entry/ Фіксація поточного стану do/ Копіювання БД	Звичайний
Помилка (Error)	entry/ Перевірка цілісності копії do/ Відображення повідомлення exit/ Сповіщення адміністратора	Звичайний
Завершення (Shutdown)	entry/ Збереження сесій do/ Закриття з'єднань exit/ Запис у системний лог	Кінцевий

Переходи:

З стану	До стану	Тригер [умова] / Дія
Вимкнена	Ініціалізація	startSystem() / Завантаження модулів
Ініціалізація	Очікування	[connectionOK] / Відображення екрану входу
Ініціалізація	Помилка	[connectionFailed] / Вивести помилку БД
Очікування	Обробка запиту	userAction() / Визначення типу операції
Очікування	Резервне копіювання	timerFired()[hour==3:00] / Запуск резервування
Очікування	Очікування	deadlineCheck()[строк < 6 год] / Надіслати сповіщення ← рефлексивний
Обробка запиту	Очікування	[operationComplete] / Оновити UI
Обробка запиту	Помилка	[operationFailed] / Rollback транзакції
Формування звіту	Очікування	[reportGenerated] / Показати попередній перегляд
Резервне копіювання	Очікування	[backupComplete] / Записати час копії

З стану	До стану	Тригер [умова] / Дія
Помилка	Очікування	retryAction()[errorResolved] / Відновити роботу
Будь-який	Завершення	shutdownCommand() / Зберегти стан

Діаграма 2.2 — Стани справи про затримання (DetentionRecord)

Діаграма описує повний цикл існування облікової справи особи в ізоляторі.

Стан	Внутрішні дії	Тип
Нова (New)	entry/ Присвоєння номера справи	Початковий
	entry/ Генерація штрих-коду	
Активна (Active)	entry/ Запуск таймера строку do/ Моніторинг граничного часу do/ Оновлення індикатора кольору	Звичайний
Продовжена (Extended)	entry/ Оновлення граничного часу entry/ Запис реквізитів рішення суду do/ Моніторинг нового строку	Звичайний
Підготовка до звільнення (Releasing)	entry/ Перевірка переліку майна do/ Формування документів	Звичайний
Звільнена (Released)	entry/ Оновлення підпису звільнення do/ Формування підпису звільнення entry/ Звільнення місця в камері exit/ Архівування картки	Звичайний
Передана (Transferred)	entry/ Формування супровідного листа entry/ Оновлення статусу exit/ Сповіщення установи-отримувача	Звичайний
Архівна (Archived)	entry/ Переміщення до архіву БД do/ Недоступна для редагування	Кінцевий

Переходи:

З стану	До стану	Тригер [умова] / Дія
Нова	Активна	confirm() / Розмістити особу в камері
Активна	Активна	deadlineWarning()[залишок < 6 год] / Сповіщення чергового ← рефлексивний
Активна	Продовжена	extendDetention(doc) [рішення суду наявне] / Оновити deadline
Продовжена	Продовжена	deadlineWarning()[залишок < 1 год] / Критичне сповіщення ← рефлексивний
Активна	Підготовка до звільнення	do initiateRelease() / Перевірити майно
Продовжена	Підготовка до звільнення	do initiateRelease() / Перевірити майно
Підготовка до звільнення	Звільнена	confirmRelease()[майно повернуто] / Закрити справу
Підготовка до звільнення	Передана	transfer(destination) / Сформувати супровідний лист
Активна	Передана	immediateTransfer() / Екстрена передача
Звільнена	Архівна	archiveRecord() / Перемістити до архіву
Передана	Архівна	archiveRecord() / Перемістити до архіву

Діаграма 2.3 — Стани камери (Cell)

Діаграма відображає зміну стану камери ізолятора залежно від кількості утримуваних та технічних обставин.

Стан	Внутрішні дії	Тип
Вільна (Free)	entry/ Оновлення заповненості: 0 do/ Камера доступна для розміщення	Початковий
Часткова зайнятість (Partial)	entry/ Оновлення лічильника do/ Відображення зеленого індикатора	Звичайний
Повністю зайнята (Full)	entry/ Блокування розміщення do/ Відображення червоного індикатора do/ Моніторинг строків по камері	Звичайний
Технічне обслуговування (Maintenance)	entry/ Переведення осіб до інших камер do/ Недоступна для розміщення exit/ Перевірка готовності	Звичайний
На перевірці (Inspection)	entry/ Фіксація часу початку перевірки do/ Обмежений доступ exit/ Запис результату перевірки	Звичайний
Закрита (Closed)	entry/ Архівування даних камери do/ Недоступна	Кінцевий

Переходи:

З стану	До стану	Тригер [умова] / Дія
Вільна	Часткова зайнятість	addDetainee()[occupancy > 0 && < capacity] / Оновити лічильник
Часткова зайнятість	Повністю зайнята	addDetainee()[occupancy == capacity] / Заблокувати
Повністю зайнята	Часткова зайнятість	removeDetainee()[occupancy < capacity] / Розблокувати
Часткова зайнятість	Вільна	removeDetainee()[occupancy == 0] / Оновити статус
Повністю зайнята	Вільна	clearCell() / Переведення всіх осіб
Будь-який	Технічне обслуговування	scheduleMaintenance() / Відкрити наряд
Технічне обслуговування	Вільна	maintenanceComplete()[перевірку пройдено] / Відновити доступ
Часткова зайнятість	На перевірці	startInspection() / Призупинити розміщення
На перевірці	Часткова зайнятість	inspectionComplete() / Відновити роботу
На перевірці	На перевірці	logInspectionNote() / Додати нотатку ← рефлексивний
Технічне обслуговування	Закрита	closePermanently() / Архівувати камеру

Частина 3. Діаграми діяльності (Activity Diagrams)

Діаграма 3.1 — Реєстрація затриманого (UC-1)

Swimlanes: Черговий ІТУ | Система | База даних

№	Діяльність	Swimlane	Тип вузла
●	Початок	—	Початковий стан
1	Відкрити форму реєстрації нової особи	Черговий ІТУ	Діяльність
2	Ввести особисті дані та завантажити фото	Черговий ІТУ	Діяльність
◇	Перевірити дублікати в базі даних?	Система	Рішення
3a	[Дублікат знайдено] Відобразити попередження та картку існуючої особи	Система	Діяльність
3b	[Дублікатів немає] Перейти до кроку 2	Система	Діяльність
═	СИНХРОНІЗАЦІЯ (fork): паралельне виконання	Система	Синхронізація
4a	Перевірити коректність документа	Система	Діяльність
4b	Зберегти фото на сервері	База даних	Діяльність
═	СИНХРОНІЗАЦІЯ (join): об'єднання гілок	Система	Синхронізація
5	Ввести підставу затримання та обрати строк	Черговий ІТУ	Діяльність
6	Розрахувати граничний строк утримання	Система	Діяльність
◇	Вибрати камеру для розміщення?	Черговий ІТУ	Рішення
7a	[Камера доступна] Призначити особу до камери	Система	Діяльність
7b	[Камера переповнена] Вивести попередження	Система	Діяльність
8	Згенерувати номер справи та штрих-код	Система	Діяльність
9	Зберегти картку до бази даних	База даних	Діяльність
10	Відобразити особу в таблиці з зеленим індикатором	Система	Діяльність
●	Кінець	—	Кінцевий стан

Діаграма 3.2 — Контроль строків утримання (UC-2)

Swimlanes: Система (автоматично) | Черговий ІТУ | Начальник зміни

№	Діяльність	Swimlane	Тип вузла
●	Початок (таймер кожні 60 сек)	—	Початковий стан
1	Отримати список активних справ з БД	Система	Діяльність
2	Розрахувати залишок строку для кожної особи	Система	Діяльність
═	СИНХРОНІЗАЦІЯ (fork): паралельна перевірка кожної справи	Система	Синхронізація
◇	Залишок строку?	Система	Рішення
3а	[> 6 год] Встановити зелений індикатор	Система	Діяльність
3б	[1-6 год] Встановити жовтий індикатор та сповістити	Система	Діяльність
3в	[< 1 год] Встановити червоний індикатор + звуковий сигнал	Система	Діяльність
3г	[= 0] Фіксувати перевищення строку в аудиті	Система	Діяльність
═	СИНХРОНІЗАЦІЯ (join): об'єднання результатів	Система	Синхронізація
4	Оновити таблицю на екрані чергового	Система	Діяльність
◇	Черговий отримав критичне сповіщення?	Черговий ІТУ	Рішення
5а	[Так] Переглянути картку особи	Черговий ІТУ	Діяльність
◇	Рішення чергового?	Черговий ІТУ	Рішення
6а	[Звільнити] Ініціювати оформлення звільнення	Черговий ІТУ	Діяльність
6б	[Продовжити строк] Ввести реквізити рішення суду	Черговий ІТУ	Діяльність
7	Система оновлює граничний строк	Система	Діяльність
8	Записати зміну до журналу аудиту	Система	Діяльність
5б	[Сповіщення підтверджено без дій] Чекати наступного циклу	Черговий ІТУ	Діяльність
●	Кінець циклу (запуск через 60 сек)	—	Кінцевий стан

Діаграма 3.3 — Оформлення звільнення (UC-3)

Swimlanes: Черговий ІТУ | Система | База даних | Принтер

№	Діяльність	Swimlane	Тип вузла
●	Початок	—	Початковий стан
1	Знайти особу (ПІБ або штрих-код)	Черговий ІТУ	Діяльність
2	Відкрити картку та перейти на вкладку «Вилучені речі»	Черговий ІТУ	Діяльність
═	СИНХРОНІЗАЦІЯ (fork)	Система	Синхронізація
3a	Відобразити перелік вилучених предметів	Система	Діяльність
36	Перевірити наявність усіх предметів на складі	База даних	Діяльність
═	СИНХРОНІЗАЦІЯ (join)	Система	Синхронізація
◇	Усі речі фізично наявні?	Черговий ІТУ	Рішення
4a	[Так] Позначити кожен предмет як «Повернуто»	Черговий ІТУ	Діяльність
46	[Ні] Зафіксувати розбіжність, сповістити начальника	Система	Діяльність
5	Натиснути «Оформити звільнення»	Черговий ІТУ	Діяльність
6	Обрати підставу звільнення	Черговий ІТУ	Діяльність
◇	Підстава коректна та дані повні?	Система	Рішення
7a	[Так] Змінити статус справи на «released»	База даних	Діяльність
76	[Ні] Вивести помилку та заблокувати кнопку	Система	Діяльність
8	Звільнити місце в камері, оновити заповненість	База даних	Діяльність
9	Сформувати пакет документів (акт, розписка, картка)	Система	Діяльність
◇	Принтер доступний?	Система	Рішення
10a	[Так] Надіслати документи на друк	Принтер	Діяльність
106	[Ні] Зберегти PDF на диск	Система	Діяльність
11	Зібрати підписи на роздрукованих документах	Черговий ІТУ	Діяльність
12	Архівувати картку, записати до аудиту	База даних	Діяльність
●	Кінець	—	Кінцевий стан

Частина 4. Діаграма компонентів (Component Diagram)

Система розподілена на 5 компонентів відповідно до тришарової архітектури (Presentation → Business Logic → Data Access) з двома допоміжними сервісами.

Компонент	Тип	Мова	Призначені класи
Presentation Layer (UI)	Application	TypeScript	DutyOfficer (UI)
		(React/Vue)	MedicalStaff (UI)
Business Logic Layer	DLL	Python	NotificationService
		(FastAPI)	CellAssignment
Domain Layer	DLL	Python	Person (abstract)
			RepairGenerator
			Staff (abstract)
Data Access Layer	DLL	Python	RepairOfficer<T>
		(SQLAlchemy)	MedicalStaff
Database	Database	PostgreSQL 15	Внутрішні таблиці БД:
			Patients, detentions,
			cells, events, documents, audit_log

Залежності між компонентами

Компонент А	→ залежить від	Компонент В	Причина
Presentation Layer	---->	Business Logic Layer	UI викликає API-методи бізнес-логіки
Business Logic Layer	---->	Domain Layer	Операції оперують доменними класами
Business Logic Layer	---->	Data Access Layer	Збереження та читання даних
Data Access Layer	---->	Database	SQL-запити до PostgreSQL
Presentation Layer	-->	Data Access Layer	Непряма залежність через API

Опис компонентів

1. Presentation Layer (Application, TypeScript)

Реалізує веб-інтерфейс: таблиця утримуваних, форма реєстрації, картка особи, панель сповіщень. Спілкується з Business Logic Layer через REST API (JSON). Реалізує інтерфейс INotificationService для відображення сповіщень у браузері.

2. Business Logic Layer (DLL, Python / FastAPI)

Містить усю бізнес-логіку: реєстрацію затриманих, розрахунок строків, обробку подій, формування звітів. Надає REST API для Presentation Layer та звертається до Domain Layer і Data Access Layer.

3. Domain Layer (DLL, Python)

Містить доменні класи з атрибутами та методами. Не залежить від жодного іншого компонента

системи — чистий домен без залежностей на фреймворки.

4. Data Access Layer (DLL, Python / SQLAlchemy)

Параметризований Repository<T> реалізується для кожного доменного класу. Забезпечує CRUD-операції, транзакції та журналювання всіх змін через AuditLog.

5. Database (PostgreSQL 15)

Реляційна СУБД. Містить усі таблиці системи. З'єднання лише через Data Access Layer — прямий доступ з інших компонентів заборонено.

Підсумок виконання лабораторної роботи

Вимога	Виконано	Деталі
Класів на діаграмі класів ≥ 10	✓ 15 класів	Person, Detainee, Staff, DutyOfficer, MedicalStaff, DetentionRecord, Cell, CellAssignment, Event, Document, Property, AuditLog, INotificationService, ReportGenerator, Repository<T>
Атрибутів у класі ≥ 5	✓ 5-8 атрибутів	Кожен клас має 5-8 атрибутів із типами
Операцій у класі ≥ 5	✓ 5 операцій	Кожен клас має рівно 5 операцій з параметрами та типами повернення
Всі типи відношень	✓ Усі 5 типів	Generalization, Association, Composition, Aggregation, Instantiation + Realization
Діаграм станів ≥ 3	✓ 3 діаграми	Система в цілому, DetentionRecord, Cell
Станів у діаграмі ≥ 6	✓ 7-8 станів	Кожна діаграма має 7-8 станів
Обидва типи переходів	✓ Є	Рефлексивні: 'deadlineCheck' та 'logInspectionNote'
Тригери/умови/дії	✓ Усі переходи	Кожен перехід має тригер, умову або дію
Діаграм діяльності = 3	✓ 3 діаграми	UC-1 Реєстрація, UC-2 Контроль строків, UC-3 Звільнення
Діяльностей ≥ 6	✓ 10-12	Кожна діаграма містить 10-12 вузлів
Рішення та синхронізація	✓ Є в кожній	Fork/Join синхронізація + ромб рішення у кожній діаграмі
Компонентів ≥ 3	✓ 5 компонентів	Presentation, Business Logic, Domain, Data Access, Database
Класи розподілені	✓ Усі 15	Кожен клас призначено до одного з 5 компонентів